INDEX

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1.	1/7	Introduction to basic HTML	CO1	10	
2.	8/7	Design HTML using audio	CO1	10	
3.	15/7	Design HTML using video	CO1	9	
4.	22/7	Program using JS (arrays)	CO1	10	
5.	29/7	Develop dynamic webpage	CO1	9	
6.	5/8	Implement modules in nodejs	CO2	10	
7.	12/8	Miniapplication using express and node.js	CO2	10	
8.	19/8	Code tantra - static webpage	CO1	9	
9.	2/9	code tantra - portfolio	CO1	9	
10.	9/9	code tantra - simple calculator.	CO1	10	
11.	16/9	Create a functional & Class	CO2	10	
12.	23/9	Implement http methods	CO2	10	
13.	30/9	Develop CRUD operations	CO3	10	
				10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999,8712008222. Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	3rd	**Semester:**	3-1
Name of the Student:	T. Sunitha	**Roll Number:**	A24126552183
Name of the Lab:	Full stack lab	**Timings:**	9:40 to 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4 Marks)	Results (5 Marks)	Viva-Voce (2 Marks)			
01/7	01	4	4	5	1	14		
08/7	02	4	4	5	1	14		
15/7	03	4	4	5	2	15		
22/7	04	4	4	5	2	15		
29/7	05	4	4	5	2	15		
05/8	06	4	4	5	1	14		
12/8	07	3	4	5	1	13		
19/8	08	3	4	5	1	13		
2/9	09	3	4	5	1	13		
9/9	10	3	4	5	1	13		
16/9	11	4	4	5	2	15		
23/9	12	4	4	5	2	15		
30/9	13	4	4	3	2	14		

Name of the Faculty: Prasanna Priya

Faculty Signature:

Introduction to HTML and CSS?

HTML (Hyper text Markup Language) is the standard markup language used to create and structure content on webpages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

CSS (Cascading Style Sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal or external stylesheets.

→ HTML and CSS work together to develop webpages where HTML provides the structure and content while CSS provides the styling, layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces.

Difference between MERN stack and FULL stack.

→ Mern stack is a javascript based technology stack used for developing modern web applications. MERN stands for MongoDB, express.JS, React.js, Node.js, MongoDB is used for database management, Express.js handles the backend framework and APIs, React.js is used to develop frontend user interface and Node.JS provides the server side runtime environment.

→ Full stack development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic maraging databases.

HTML components:

1. HTML Tags: Define the structure and elements of a webpage.
2. Headings - used to display titles and headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> tag.
4. Links - used to display images using <image> tag.
5. Lists - used to create ordered and unordered lists using <ol>, <ul>
6. Images - used to display images using <image> tag.
7. Tables - used to organize data in rows and columns.
8. Forms - used to collect user input using elements such as text boxes, buttons and checkboxes.

9. Div and Span - used to group and organize webpage content.

10. Semantic elements - elements such as <header>, <nav>, <section>, <articles>, <footer> provide meaningful webpage structures

CSS components:

1. Selectors - select HTML elements to apply styles
2. properties - Define the visual characteristic of elements.
3. Values - specify the setting of a CSS property
4. Colors - Define text, background, border colors
5. Fonts - control font family, size, style and weight
6. Backgrounds - set background colors and images
7. Borders - Define the border style, width and radius.
8. Margin - control the space outside an element.
9. padding - controls the arrangement of layout - elements using Flexbox, grid and positioning
10. padding - controls the space inside element.

HTML VS HTML5

HTML	HTML5
older version of HTML	Latest major version of HTML
limited Multimedia support	Supports audio and video
uses external plugins for multimedia	No external plugs needed.
Has fewer Semantic tags	Provides semantic tags
NO <canva> element	Supports <canvas>
limited web storage	Supports local storage.
complex DOCTYPE declaration	simple <! DOCTYPE HTML>

CSS VS CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS.

Basic styling features	provides advanced styling features
limited animations	supports animations and transitions
No flexible layout systems	supports flexbox and grid
limited visual effects	supports shadows, gradients, effects
No media queries	Supports media queries
uses a single specification	Divided into different modules

Design a static webpage using HTML and CSS.

Pomodoro Timer using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
        body {
            font-family : Arial;
            background : #f5f5f5;
            text-align: center;
            Padding : 50px
        }
```

```
.timer {
        width : 300px
        margin : auto
        padding : 30px
        background: white
        border-radius : 15px .
        box-shadow: 0 0 10px gray ;
}
 h1 {
    color: # e74c3c;
}
time {
    font-size: 50px
    margin : 20px
}
button {
    padding: 10px 20px ;
    margin: 5px ;
    border : none;
    border-radius : 5px;
    background : # e74c3c;
    color : white;
}
</style>
</head>
<body>
    <div class = "Timer">
        <h1> Pomodoro Timer </h1>
```

```
<div class="time">25:00</div>
<button>Start</button>
<button>Reset</button>
</div>
</body>
</html>
```

O/p:

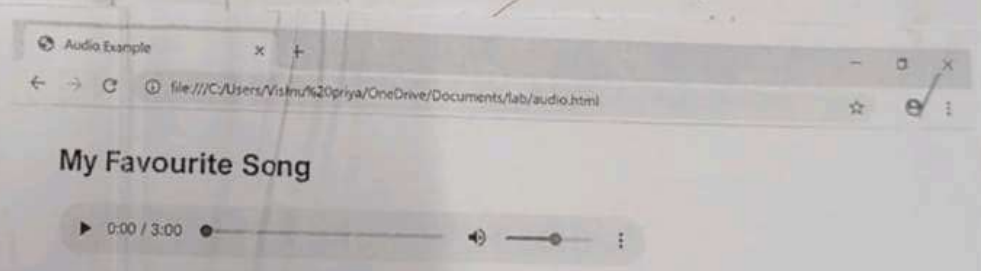


1. Design an HTML code using audio tags.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Audio Example</title>
</head>
<body>
    <h2>My favourite song</h2>
    <audio controls>
        <source src="song.mp3" type="audio/mpeg">
        Your browser does not support the audio tag
    </audio>
</body>
</html>
```

O/p: Displays an audio player with play, pause and volume controls:



Steps:

Create a new HTML file.

Add the basic HTML structure

Add heading using <h2> tag

Insert the <audio> tag with controls attribute

Specify the audio file using <source> tag

Save and open the HTML file in a web browser.

Play the audio using displayed controls.

(ii) Design a HTML code using video tags.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> video Example </title>
</head>
<body>
    <h2> My video </h2>
    <video width = "400" controls>
        <source src = "video.mp4" type =
                "video mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.



Steps:

Create a new HTML file

Add a basic HTML structure

Add a heading using the <h2> tag

Insert the <video> tag with controls attribute

Specify the video file using <source> tag with controls attribute specify the video file using <source> tag

Save the file and open it in web browser.

Play the video using displayed controls.

write a program using javascript (arrays, functions)

Code:

```
let numbers = [12,45,8,23,56,19,34,67,90,11];
function displayArray(arr) {
    console.log("Array Elements: ")
    for (let i=0; i<arr.length;i++) {
        console.log(arr[i]);
    }
}
function findSum(arr) {
    let sum = 0;
    for (let i=0; i<arr.length;i++) {
        sum += arr[i];
    }
    return sum;
}
```

O/P:-

```
12
13
14
15
16
19
24
34
67
90
99
Sum: 403
```

EXPERIMENT-04

4. Develop dynamic webpage using javascript

PROGRAM:-

```
<!DOCTYPE HTML>
<html>
<head>
<title> Dynamic webpage </title>
<style>
body {
text_align: center;
font_family : Arial;
margin_top : 100px;
}
button {
Padding : 10px 20px;
font_size : 10px;
cursor : pointer;
}
</style>
</head>
<body>
<h1 id="heading"> welcome to my
webpage </h1>
```

```
<p id = "message" > click the button to change
    this webpage </p>
<button onclick = "change page()" >
        clickMe </button>

<script>

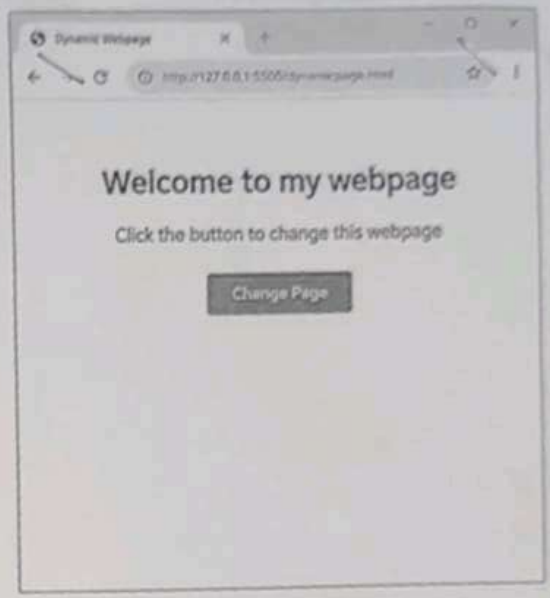
function changepage(){

  document.getElementById("heading").
        innerHtml = "welcome to JavaScript!"

document.getElementById("heading").innerHTML
    = "The webpage has been changed
      dynamically.
  document.body.style.background color= "lightblue"
}
</script>
</body>
 </html>
```

Output :-

When



Before clicking the button

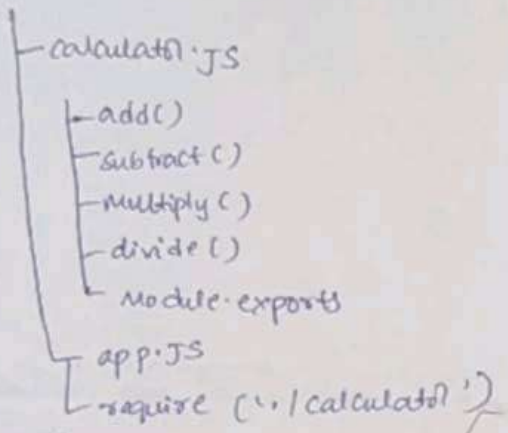


After clicking the button

EXPERIMENT-5

5. Implement modules in Node.JS.

AIM: To develop a Node.JS application using modules to perform arithmetic operation Node.JS modules.



① Calculator.JS – module.

```
function add(a,b) {
  return a+b;
}
function subtract(a,b) {
  return a-b;
}
function multiply(a,b) {
  return a*b;
}
function divide(a,b) {
  return a/b;
}
module.exports = {add, subtract, multiply, divide};
```

② app.js – main program

```
const calculator = require('./calculator');
console.log("Addition:", calculator.add(20,10));
console.log("Subtraction:", calculator.subtract(20,10));
console.log("Multiplication:", calculator.multiply(20,10));
console.log("Division:", calculator.divide(20,10));
```

③ index.html

```
<!DOCTYPE html>
<html>
<head>
<title> Node j.s Modules </title>
<style>
body {
  font-family : Arial;
  text-align : center;
  margin-top : 80px;
}
```

```
box {
 width : 400px;
 margin : auto;
 Padding : 30px;
 Border: 1px solid #ccc;
 border-radius : 10px;
}
input, select, button {
Padding : 10px;
margin : 8px;
}
button {
Cursor: pointer;
}
#result {
 margin-top: 20px;
 font-size : 20px;
}
</style>
</head>
<body>
<div class = "box">
<h> node.js modules </h1>
<input type = "number" id = "num1" placeholder
 = "first number">
<input type = "number" id = "num2" placeholder
 = "second number">
<br>
<select id = "operation">
<option value = "add"> Addition </option>
<option value = "Subtract"> Subtraction </option>
<option value = "multiply"> Multiplication </option>
<option value = "divide"> Division </option>
</select>
<br>
<button onclick = "Calculate()"> Calculate
</button>

<div id = "result"> </div>
</div>
<script>
function Calculate() {
Let a = Number(document.getElementById
("num1").value);
Let b = Number(document.getElementById)
("num2").value);
Let Operation = document.getElementById(
"operation").value;
Let result;
```

```
if (operation === "add")
result = a + b;
else if (operation === "subtract")
result = a - b;
else if (operation === "Multiply")
result = a * b;
else
result = a/b;
document.getElementById("result").inner
        HTML = "Result : " + result;
}
</script>
</body>
</html>
```

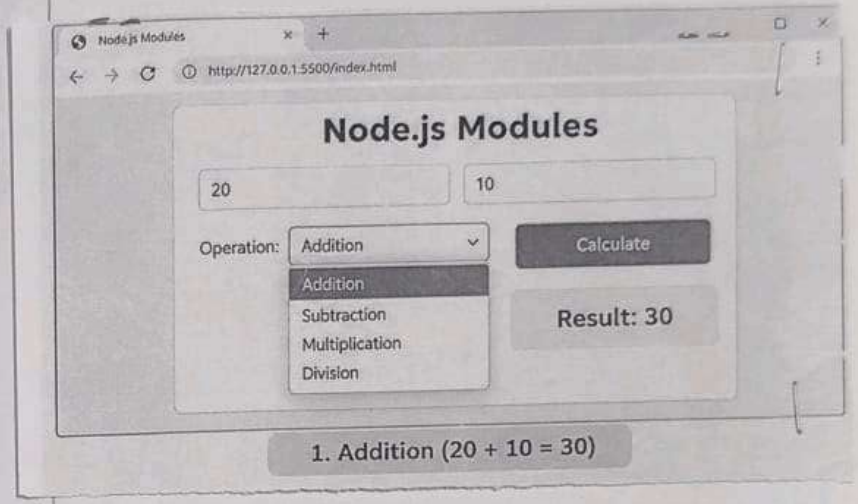
Output :-

Addition : 30

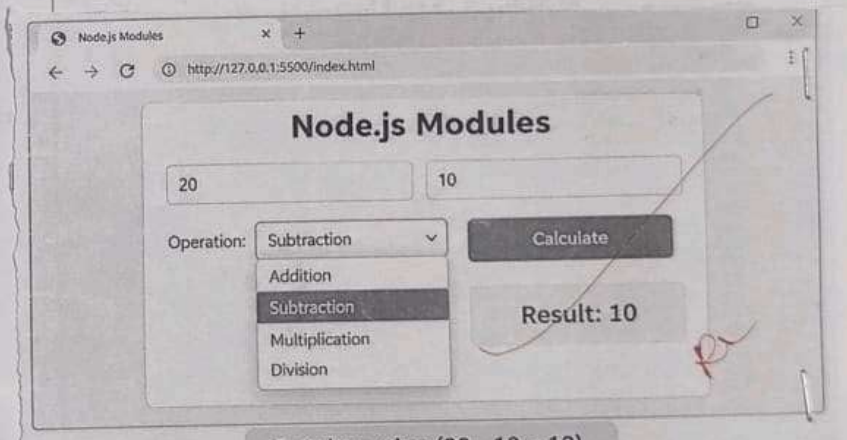
Subtraction : 10

Multiplication : 200

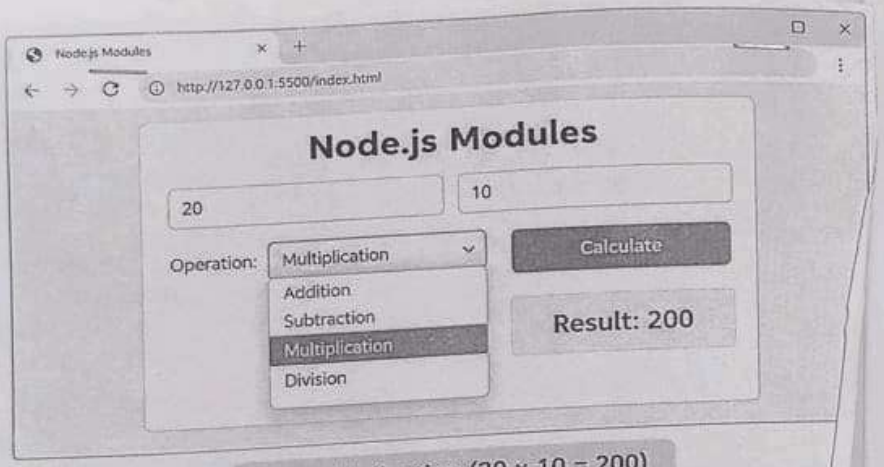
Division : 2



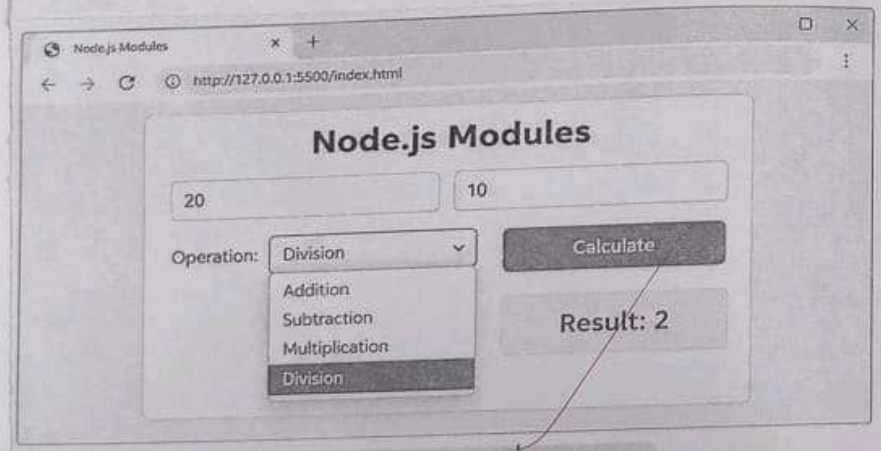
1. Addition (20 + 10 = 30)



2. Subtraction (20 - 10 = 10)



3. Multiplication (20 × 10 = 200)



4. Division (20 ÷ 10 = 2)

EXPERIMENT-6

Develop mini application using express and node.js

AIM: TO develop a mini login application using node.js and express.js that validates the username and password entered by the user.

server.js:-

```
const express = require("express");
const cors = require("cors");
const app = express();
const corsOptions = {
origin: "http://localhost:3000",
methods: ["GET", "POST", "PUT", "DELETE"],
allowedHeaders: ["Content-Type", "Authorization"]
};
app.use(cors(corsOptions));
app.use(express.json());
app.use(express.urlencoded({extended: true}));
app.use(express.static("public"));
app.post("/login", (req, res) => {
const {username, password} = req.body;
```

```
if (username === "admin" && password ===
                "1234")
{
  res.json({
    sucess: true,
    message: "login succesful"
  });
} else {
res.json({
sucess: false,
message: "invalid username or password"});
}
});
app.listen(3000, () => {
  console.log("server running at http://
          localhost:3000");

});
```

② public/index.html

```
<!DOCTYPE html>
<html>
<head>
 <title>login page</title>
 <style>
```

```
body {
font-family: Arial;
background-color: lightblue;
}
login {
 width: 300px
margin: 100px auto;
padding: 25px;
background-color: white;
text-align: center;
 border-radius: 10px;
}
input {
width: 90%;
padding: 10px;
margin: 10px;
}
 button {
padding: 10px 30px;
background-color: blue;
colo: white;
border: none;
cursor: pointer;
}
```

```
#message {
  margin-top: 15px;
}
</style>
</head>
<body>
<div class = "login">
<h2> login </h2>
<input type = "text" id = "username" placeholder =
    "Enter username">
<input type = "password" id = "password" placeholder
    = "enter password">
<button onclick = "login()"> login </button>
<p id = "message"> </p>
</div>
<script>
function login(){
Const username = document.getElementById
    ("username").value;
Const password = document.getElementById
  ("password").value
fetch ("http://localhost:3000/login", {
  method : "POST",
  headers : {
"content-Type" : "application/json" },
body : JSON.stringify ({
Username : username,
password: password
})
})
.then(response => response.json())
.then( data => {
document.getElementById("message").textContent
    = data.message;
})
.catch (error => { console.log(error);
});
}
</script>
</body>
</html>
```

folder Structure:-

```
login/
  ├── server.js
  ├── package.json
  ├── package-lock.json
  ├── node_modules/
  └── public/index.html/
```

Execution :-

Install the required packages :

```
npm init -y
npm install express cors
```

Run the server :

```
node server.js
```

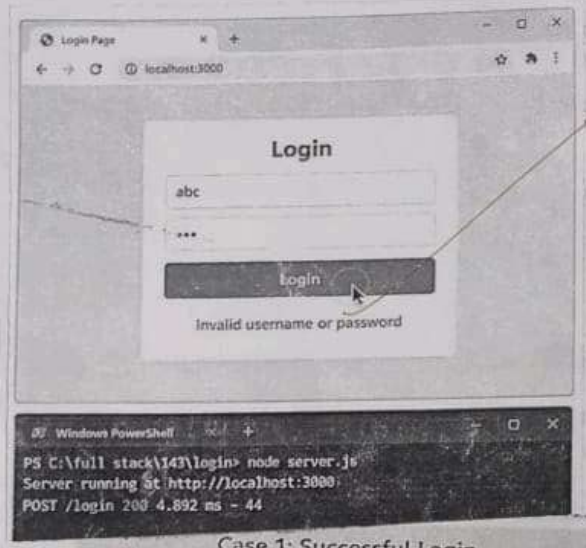
Output in the terminal :

```
Server running at http://localhost:3000
```

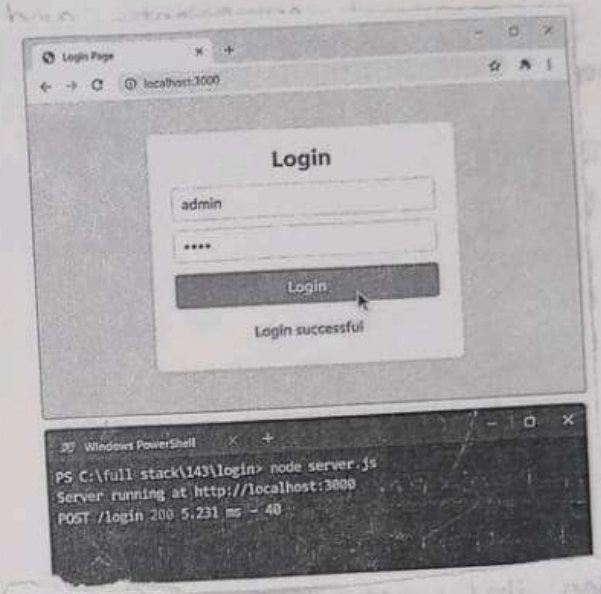
Open the browser :

```
http://localhost:3000
```

OUTPUT :-



Case 1: Successful Login
(Username: admin | Password: 1234)
Cursor is on the Login button (hover state).



Case 2: Invalid Login
(Username: abc | Password: 123)
Cursor is clicking the Login button (clicked state).

EXPERIMENT-7:

Create a functional component and class component using react.

AIM:- To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

- **Functional component:-**

A functional component is a JavaScript function that returns JSX to display the user interface. It uses React Hooks such as useState to manage state and update the webpage dynamically.

PROGRAM:-

```
import React, {useState} from
'react';
function StudentFunctional() {
    const [name, setName] = useState("Sunitha");
    const [course, setCourse] = useState("CSM");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Teju");
    };
    const changeCourse = () => {
        setCourse("CSD");
    };
    const incrementCount = () => {
        setCount(count+1);
    };
return (
    <div style={{
        border: "2px solid blue",
        Padding: "20px",
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1> Student Details </h1>
    <h2> Functional Component </h2>
    <p> <b> Name: </b> {name} </p>
    <p> <b> course: </b> {course} </p>
    <p> <b> Button clicks: </b> {count} </p>
    <button onClick={changeName}>
        changeName </button>
    <button onClick={changeCourse}> Change
        Course </button>
    <button onClick={incrementCount}> Increment
        </button>
```

```
</div>
);
```

Outputs:-

When the user clicks the buttons, the name, course and count change dynamically without reloading the webpage.

- Class component:-

A class component is an ES6 JavaScript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState()

Program:-

```
import React, {Component} from
'react';
class StudentClass extends Component {
    constructor(props) {
        super(props);
        this.state = {
            name: "Sunitha",
            course: "CSM",
            count: 0
        };
    }
    changeName = () => {
        this.setState({
            name: "Suni"
        });
    };
    changeCourse = () => {
        this.setState({
            course: "CSM"
```

```
});
};
incrementCount = () => {
  this.setState((prevState) => {
```

```
changeName </button>
<button onClick={this.changeCourse}>
  changeCourse </button>
<button onClick={this.incrementCount}>
  Increment </button>
</div>
```

React App

localhost:3000

React Components

Student Details

Functional Component

Name: Tejaswari
Course: CSE
Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari
Course: CSE
Button Clicks: 0

Change Name | Change Course | Increment

EXPERIMENT-8:

Implement http methods using JSON server.

AIM: To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE. Using JSON Server to perform CRUD (Create, read, update and delete) operations on student records.

THEORY:

JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a server.

- GET - Retrieves data from the server.
- POST - Adds new data to the server.
- PUT - Replaces an existing record.
- PATCH - Updates selected fields of a record.
- DELETE - Removes a record from the server.

a. Create a JSON server:

Step-1: Initialize the project.

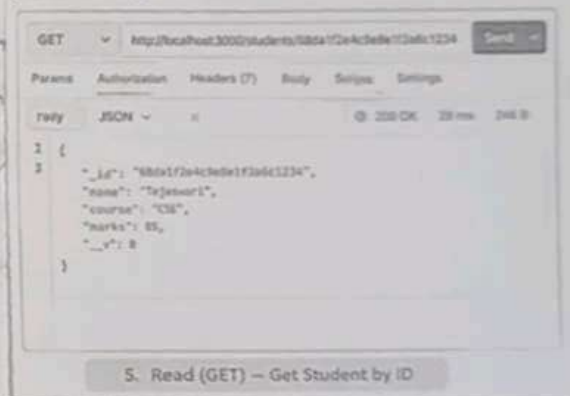
open the vs code terminal and execute the following commands.

```
mkdir JSONserver
cd JSONServer
npm init -y.
npm install json server
```

Step 2:- create db.json

create a file named db.json and add the following student data.

```
{
  "students": [
    {
      "id": "1",
      "name": "Sunitha",
      "course": "CSM",
      "masks": "90"
    },
    {"id": "2"
    "name":
    "course"
    "marks"
  }
]
  "id":
  "name":
  "course":
  "marks": 88
```



Step 3: Start the JSON server.

For the current JSON server version,

Execute:

npx json-server db.json

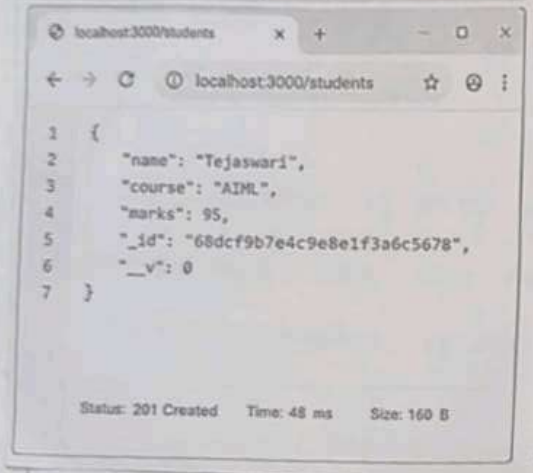
The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

3. POST (Create Student) – Using Postman



method:

thod is used to.

from JSON server.

ost:3000/students')

.then(response => response.json())

.then(data => {

console.log(data);

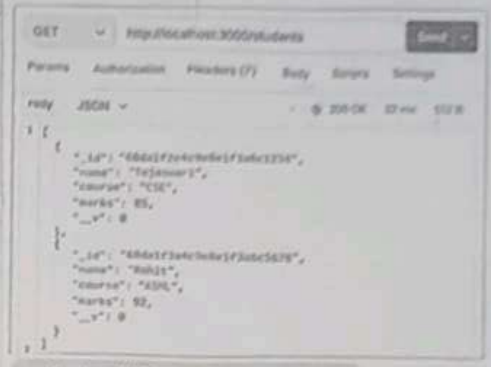
})

.catch(error => {

Console.log(error);

});

OUTPUT:-



4. Read (GET) – Get All Students

B POST Method:- The POST method is used to insert a new student record into the server.

code: fetch("http://localhost:3000/students", {

method: 'POST',

header: {

'content-type': 'application/json'

},

body: JSON.stringify({

name: 'Rahul',

course: 'CSE',

marks: 92

})

})

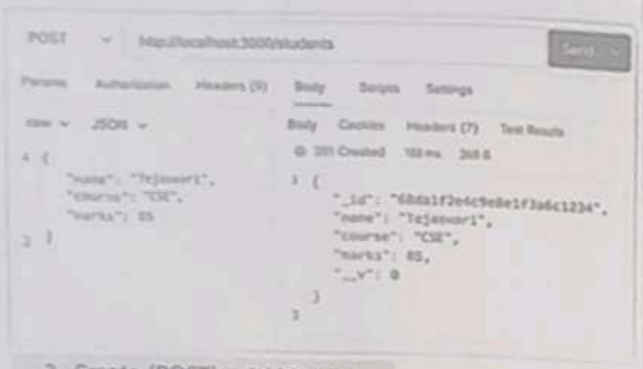
.then(response => response.json())

.then(data => {

console.log(data);

```
})
.catch(error => {
    console.log(error);
});
```

Output:



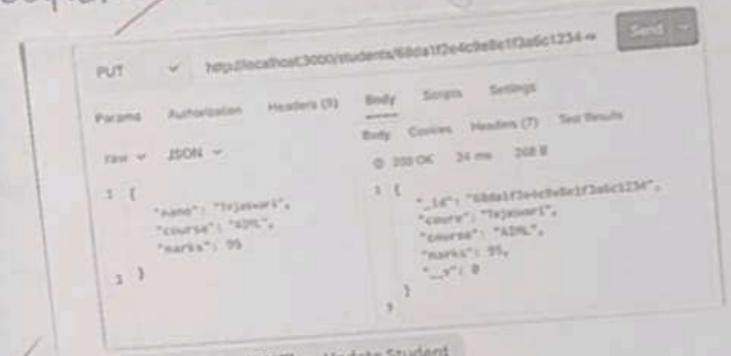
3. Create (POST) – Add Student

C. PUT METHOD: The put method is used to replace an existing student record with updated data.

Code:

```
fetch('http://localhost:3000/students/1', {
    method : 'PUT',
    headers : {
      'Content-TYPE' : 'application/json'
    },
    body : JSON.stringify ({
        name : 'sunitha',
        course : 'AIML',
        marks : 95
    })
})
.then (response => response.json())
.then(data => {
    console.log(data);
})
.catch (error => {
    console.log (error);
});
```

Output:-



6. Update (PUT) – Update Student

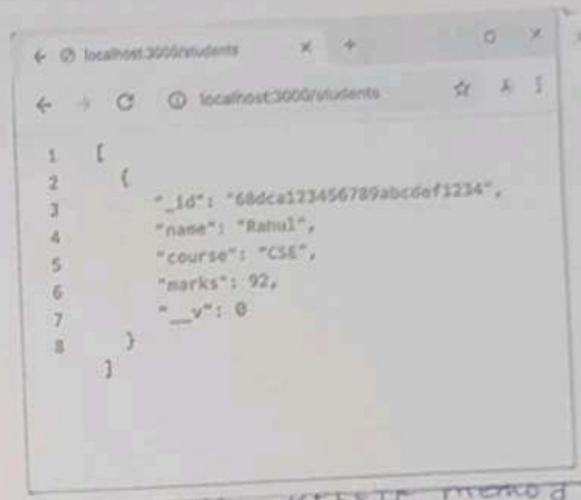
D. PATCH Method :

The PATCH method is used to update only selected fields of an existing student record.

code :-

```
fetch('http://localhost:3000/students/2', {
    method : 'PATCH',
    headers : {
        'content_TYPE': 'application/json
    },
    body : JSON_stringify({
        marks: 96
    })
})
.then (response => response.json())
.then (data => {
    CONSOLE.log(data);
})
.catch(error => {
    console.log(error);
});
```

OUTPUT :-



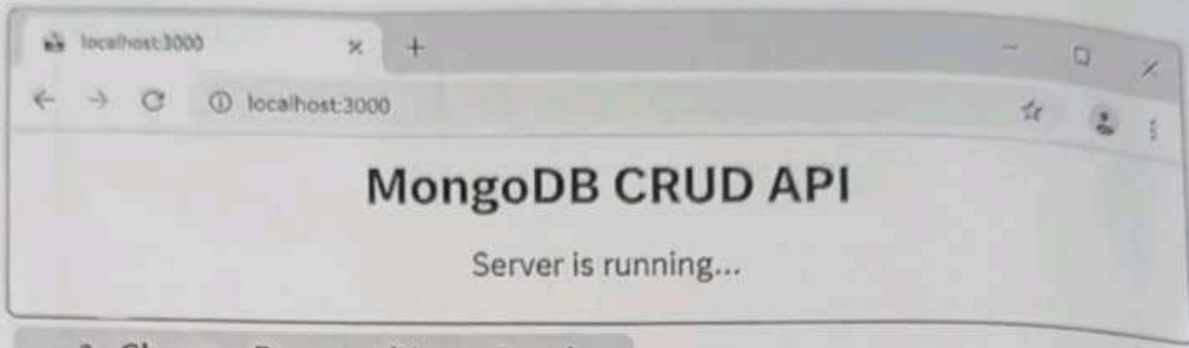
E. DELETE

The DELETE method is used to remove to existing student record from the server.

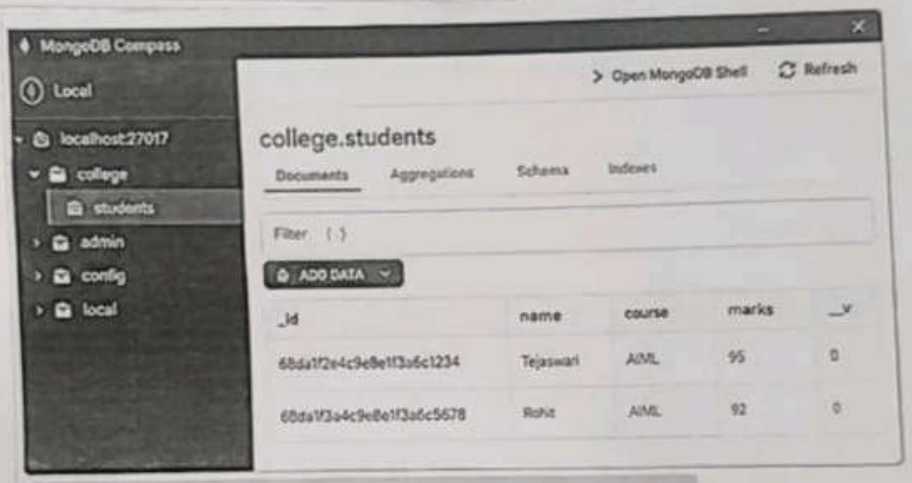
code : fetch ('http://localhost:3000/students/3', {

```
        method : 'DELETE'
    })
.then(response => {
    if (response.ok){
        console.log('student deleted
                    successfully');
    }
})
.catch (error => {
    console.log(error)
});
```

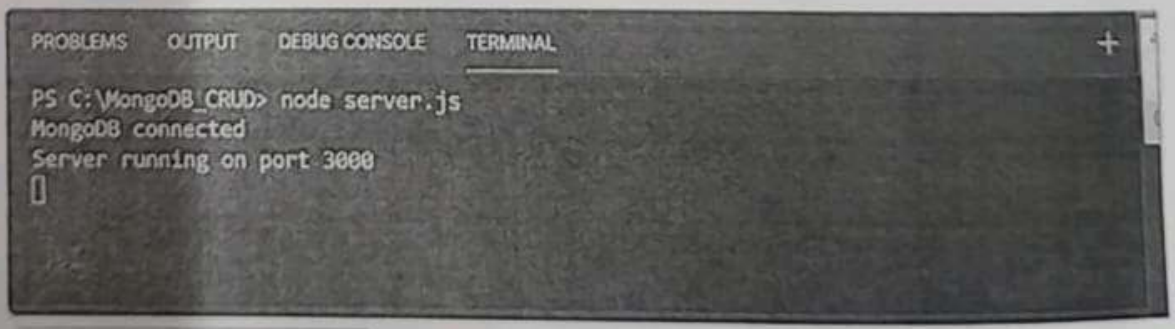
Output:-



2. Chrome Browser (Home Page)



8. MongoDB Compass – Database View

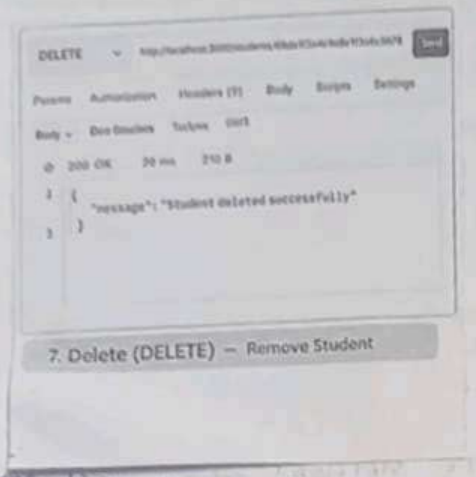


1. Terminal Output

```
});
//update
app.put('/students/:id', async (req, res) => {
  try {
    const student = await
Student.findByIdandupdate(
      req.params.id
      req.body
      { new: true, runValidators: true }
    );
if (!student) {
  return res.status(404).json({
      message: "student not found"
    });
  }
  res.status(200).json(student);
} catch (error) {
  res.status(400).json({message
    message});
  }
});
// DELETE.
app.delete('/students/:id', async
      (req, res) => {
  try {
    const student = await student.
    findByIdandDelete(
      req.params.id
    );
if(!student) {
  return res.status(404).json({
      message: "student not found"
    });
}
res.status(200).json({
  message: "student deleted successfully"
  });
} catch(error) {
    res.status(400).json({message:
      error.message});
}
});
```

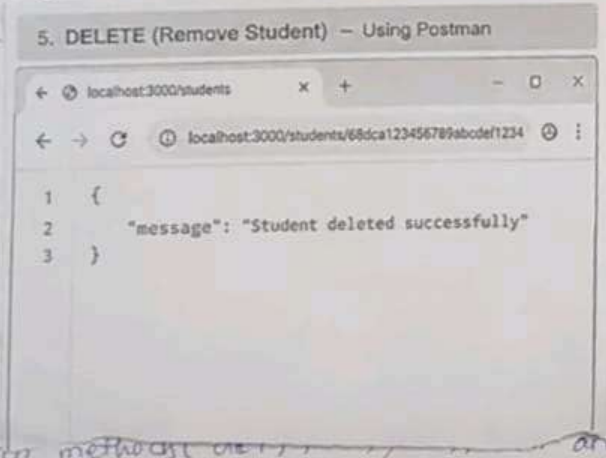
```
const app = express();
const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:
                  27017/College')
.then(() => console.log("MONGODB connected"))
.catch(err => console.log(err));
// Create Student
const studentSchema = new
mongoose.Schema({
    name: {
      type: String
      required: true
    }
});
//Create Student model
  const Student;
      mongoose.model('Student', StudentSchem
//Create Add
app.post('/students', async(req, res) => {
   try {
   const student = new Student(req.bod
   const savedStudent = await Student
                        .save();
res.status(201).json(savedStudent);
} catch(err) {
  res.status(400).json({message: err.message});
  }
});
// READ. Get all students
app.get('/[students]', async(req, res) => {
   try {
     const students = await
    Student.find();
      res.status(500).json({ message: err.
                           message });
   }
});
// READ : GET student by ID
app.get('/[students]/:id', async(req,res) => {
   try{
     Const student = await
     Student.findById(req.params.id);
     if(!student){
          return res.status(404).json({
            message: "student not found"
     });
     res.status(200).json(student)
   } catch(err) {
     res.status(400).json({message:
     err.message});
```

OUTPUT:-



• After executing the POST, PUT, PATCH and DELETE operations in sequence the remaining student will be.

OUTPUT:-



Thus, HTTP methods GET, ... and DELETE were successfully implemented using JSON server the CRUD operations were performed on student records the data was retrieved, inserted, updated and deleted using using HTTP requests.

EXPERIMENT-9

9. Develop CRUD operations using MongoDB.

AIM: To develop a web application using Node.js, express.js and Mongo DB to Perform CRUD operations (create, read, update, delete) on student records.

THEORY:-

Mongo DB is a NoSQL database that stores data in flexible JSON like documents, CRUD operations are used to manage data in a database.

Mongoose is an object data Modelling (ODM) library for Mongo DB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and project setup:

```
mkdir mongoDB-CRUD
Cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

PROGRAM:

Server.js

```
Const express = require('express');
Const mongoose = require('mongoose');
```